

UNIT I

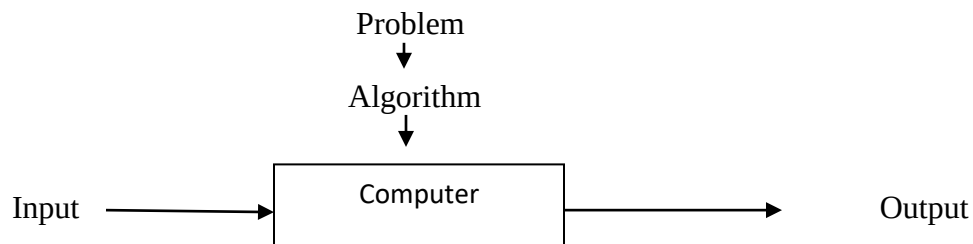
Algorithms: Definitions and notations: standard notations - asymptotic notations – worst case, best case and average case analysis; big oh, small oh, omega and theta notations; Analysis of Sorting and Searching: Heap, shell, radix, insertion, selection and bubble sort; sequential, binary and Fibonacci search. Recursive algorithms, analysis of non-recursive and recursive algorithms, solving recurrence equations, analyzing control structures.

2 Marks

1. What is an algorithm? (UQ April 2012 & APRIL 2013/2015,Nov'16)

An algorithm is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in finite amount of time.

2. Give the diagram representation of Notion of algorithm.



3. Why is the need of studying algorithms?

From a practical standpoint,

- ✓ A standard set of algorithms from different areas of computing must be known
- ✓ In addition to be able to design them
- ✓ Analyze the algorithm efficiencies.

From a theoretical standpoint, the study of algorithms is the basis of computer science.

4. What is algorithmic?

The study of algorithms is called algorithmic. It is more than a branch of computer science. It is the core of computer science and is said to be relevant to most of science, business and technology.

5. What is the formula used in Euclid's algorithm for finding the greatest common divisor of two numbers?

Euclid's algorithm is based on repeatedly applying the equality

$\text{Gcd}(m,n)=\text{gcd}(n,m \bmod n)$ until $m \bmod n$ is equal to 0, since $\text{gcd}(m,0)=m$.

6. What are the three different algorithms used to find the gcd of two numbers?

The three algorithms used to find the gcd of two numbers are .

- Euclid's algorithm
- Consecutive integer checking algorithm
- Middle school procedure

7. What are the fundamental steps involved in algorithmic problem solving?

The fundamental steps are

1. Understanding the problem.
2. Ascertain the capabilities of computational device
3. Choose between exact and approximate problem solving.

4. Decide on appropriate data structures.
5. Algorithm design techniques
6. Methods for specifying the algorithm
7. Proving an algorithms correctness
8. Analyzing an algorithm.
9. Coding an algorithm

8. What is an algorithm design technique?

An algorithm design technique is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

9. What is pseudo code?

A pseudo code is a mixture of a natural language and programming language constructs to specify an algorithm. A pseudo code is more precise than a natural language and its usage often yields more concise algorithm descriptions.

10. What are the types of algorithm efficiencies?

The two types of algorithm efficiencies are

- Time efficiency: indicates how fast the algorithm runs.
- Space efficiency: indicates how much extra memory the algorithm needs

11. Mention some of the important problem types?

Some of the important problem types are as follows.

1. Sorting.
2. Searching
3. String processing.
4. Graph problems
5. Combinatorial problems
6. Geometric problems.
7. Numerical problems

12. What are the classical geometric problems?

The two classic geometric problems are ,

1. The closest pair problem: given n points in a plane find the closest pair among them
2. The convex hull problem: find the smallest convex polygon that would include all the points of a given set.

13. What are the steps involved in the analysis framework?

The various steps are as follows

- Measuring the input's size.
- Units for measuring running time.
- Orders of growth.
- Worst case, best case and average case efficiencies

14. What is the basic operation of an algorithm and how is it identified?

The most important operation of the algorithm is called the basic operation of the algorithm, the operation that contributes the most to the total running time. It can be identified easily because it is usually the most time consuming operation in the algorithm's innermost loop.

15. What is the running time of a program implementing the algorithm?

The running time $T(n)$ is given by the following formula

$$T(n) \approx c_{op}C(n)$$

C_{op} is the time of execution of an algorithm's basic operation on a particular computer and $C(n)$ is the number of times this operation needs to be executed for the particular algorithm.

16. What are exponential growth functions?

The functions 2^n and $n!$ are exponential growth functions, because these two functions grow so fast that their values become astronomically large even for rather smaller values of n .

17. What is worst-case efficiency?(UQ APRIL 2015)

The worst-case efficiency of an algorithm is its efficiency for the worst-case input of size n , which is an input or inputs of size n for which the algorithm runs the longest among all possible inputs of that size.

18. What is best-case efficiency? (UQ APRIL 2013/2015)

The best-case efficiency of an algorithm is its efficiency for the best-case input of size n , which is an input or inputs for which the algorithm runs the fastest among all possible inputs of that size.

19. What is average case efficiency?

The average case efficiency of an algorithm is its efficiency for an average case input of size n . It provides information about an algorithm's behavior on a "typical" or "random" input.

20. What is amortized efficiency?

In some situations a single operation can be expensive, but the total time for the entire sequence of n such operations is always significantly better than the worst case efficiency of that single operation multiplied by n . This is called amortized efficiency.

21. Define O -notation?

A function $t(n)$ is said to be in $O(g(n))$, denoted by $t(n) \in O(g(n))$, if $t(n)$ is bounded above by some constant multiple of $g(n)$ for all large n , i.e., if there exists some positive constant c and some nonnegative integer n_0 such that

$$T(n) \leq cg(n) \text{ for all } n \geq n_0$$

22. Define Ω -notation?

A function $t(n)$ is said to be in $\Omega(g(n))$, denoted by $t(n) \in \Omega(g(n))$, if $t(n)$ is bounded below by some constant multiple of $g(n)$ for all large n , i.e., if there exists some positive constant c and some nonnegative integer n_0 such that

$$T(n) \geq cg(n) \text{ for all } n \geq n_0$$

23. Define Θ -notation?

A function $t(n)$ is said to be in $\Theta(g(n))$, denoted by $t(n) \in \Theta(g(n))$, if $t(n)$ is bounded both above & below by some constant multiple of $g(n)$ for all large n , i.e., if there exists some positive constants c_1 & c_2 and some nonnegative integer n_0 such that

$$c_2g(n) \leq t(n) \leq c_1g(n) \text{ for all } n \geq n_0$$

24. What are the basic asymptotic efficiency classes?

The various basic efficiency classes are.

- Constant : 1
- Logarithmic: $\log n$.
- Linear : n
- N-log-n : $n \log n$
- Quadratic : n^2
- Cubic: n^3 .
- Exponential : 2^n
- Factorial: $n!$

25. Give an non-recursive algorithm to find out the largest element in a list of n numbers.

ALGORITHM *MaxElement*($A[0..n-1]$)

//Determines the value of the largest element in a given array

//Input: An array $A[0..n-1]$ of real numbers

//Output: The value of the largest element in A

```

    maxval  $\leftarrow$  a[0]
    for I  $\leftarrow$  1 to n-1 do
    if  $A[I] > \text{maxval}$ 
        maxval  $\leftarrow$   $A[I]$ 
    return maxval

```

26. Write the general plan for analyzing the efficiency for non-recursive algorithms.

The various steps include.

- Decide on a parameter indicating input's size.
- Identify the algorithms basic operation. .
- Check whether the number of times the basic operation is executed depends on size of input. If it depends on some additional property the worst, average and best-case efficiencies have to be investigated separately.
- Set up a sum expressing the number of times the algorithm's basic operation is executed.
- Using standard formulas and rules of manipulation find a closed-form formula for the count or at least establish its order of growth.

27. Give a non-recursive algorithm for element uniqueness problem.

ALGORITHM *UniqueElements*($A[0..n-1]$)

//Checks whether all the elements in a given array are distinct

//Input: An array $A[0..n-1]$

//Output Returns 'true' if all elements in A are distinct and 'false'

//otherwise

```

    for I  $\leftarrow$  0 to n-2 do
    for j  $\leftarrow$  I+1 to n-1 do
    if  $A[I] = A[j]$  return false
    return true

```

28. Mention the non-recursive algorithm for matrix multiplication?

ALGORITHM *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)

```
//Multiplies two square matrices of order n by the definition based
//algorithm
//Input: Two n-by-n matrices A and B
//Output: Matrix C = AB
    for I ← 0 to n-1 do
    for j ← 0 to n-1 do
    C[I,j] ← 0.0
    for k ← 0 to n-1 do
    C[I,j] ← C[I,j] + A[I,k]*B[k,j]
    return C
```

29. Write a non-recursive algorithm for finding the number of binary digits for a positive decimal integer.

```
ALGORITHM Binary(n)
// Input A positive decimal integer n
// Output The number of binary digits in n's binary representation
    count ← 1
    while n>1 do
    count ← count + 1
    n ← n/2
    return count
```

30. Write a recursive algorithm to find the n-th factorial number.

```
ALGORITHM F(n)
// Computes n! Recursively
// Input A non-negative integer n
// Output The value of n!
    if n=0 return 1
    else return F(n-1) * n
```

31. What is the recurrence relation to find out the number of multiplications and the initial condition for finding the n-th factorial number?

The recurrence relation and initial condition for the number of multiplications is
 $M(n)=M(n-1)+1$ for $n>0$
 $M(0)=0$

32. Write the general plan for analyzing the efficiency for recursive algorithms.

The various steps include

- Decide on a parameter indicating input's size.
- Identify the algorithms basic operation. .
- Check whether the number of times the basic operation is executed
- Depends on size of input. If it depends on some additional property the worst, average and best-case efficiencies have to be investigated separately.
- Set up a recurrence relation with the appropriate initial condition, for the number of times the basic operation is executed.
- Solve the recurrence or at least ascertain the orders of growth of its solution.

33. Write a recursive algorithm for solving Tower of Hanoi problem.

ALGORITHM

- To move $n > 1$ disks from peg1 to peg3, with peg2 as auxiliary, first move recursively $n-1$ disks from peg1 to peg2 with peg3 as auxiliary.
- .Then move the largest disk directly from peg1 to peg3.
- Finally move recursively $n-1$ disks from peg2 to peg3 with peg1 as auxiliary.
- If $n=1$ simply move the single disk from source peg to destination peg.

34. Write a recursive algorithm to find the number of binary digits in the binary representation of an integer.

```
ALGORITHM BinRec(n)
// Input A positive decimal integer n
// Output The number of binary digits in n's binary representation
if n=1 return 1
else return BinRec(n/2)+1
```

35. What is selection sort?

Selection sort is started by scanning the entire list to find the smallest element and exchange it with the first element, putting the first element in the final position in the sorted list. Then the scan starts from the second element to find the smallest among $n-1$ elements and exchange it with the second element.

36. Mention the pseudo code for selection sort.

```
ALGORITHM SelectionSort(A[0..n-1])
//The algorithm sorts a given array by selection sort
//Input: An array A[0..n-1] of orderable elements
//Output: Array A[0..n-1] sorted in ascending order
for I ← 0 to n-2 do
  min ← I
  for j ← I+1 to n-1 do
    if A[j] < A[min] min ← j
  swap A[I] and A[min]
```

37. What is bubble sort?

Another brute force approach to sort a problem is to compare adjacent elements of the list and exchange them if they are out of order, so we end up “bubbling up” the largest element to the last position in the list. The next pass bubbles up the second largest element, and so on until $n-1$ passes, the list is sorted. Pass I can be represented as follows

```
For i = 1 to n
  For j = 1 to n
    If (a[i]>a[j])
    {
      t= a[i];
      a[i]=a[j];
      a[j]=t;
    }
```

38. Give an algorithm for bubble sort?

```

ALGORITHM BubbleSort(A[0..n-1])
//The algorithm sorts array A[0..n-1] by bubble sort
//Input: An array A[0..n-1] of orderable elements
//Output: Array A[0..n-1] sorted in ascending order
    For i = 1 to n
        For j = 1 to n
            If (a[i]>a[j])
            {
                t= a[i];
                a[i]=a[j];
                a[j]=t;
            }

```

39. Explain about the enhanced version of sequential search.

Sequential search simply compares successive elements of a list with a given search key until either a match is encountered or the list is exhausted without finding a match. The enhancement in this version is to append the search key to the end of the list, then the search for the key will have to be successful & so we can eliminate a check for the list's end on each iteration.

40. What is binary search? (UQ April'12/APRIL 2015)

If 'q' is always chosen such that 'aq' is the middle element(that is, $q = \lfloor (n+1)/2 \rfloor$), then the resulting search algorithm is known as binary search.

41. What is insertion sort?

Insertion sort is an application of decrease-by-one technique to sort an array A[0..n-1]. We assume that the smaller problem of sorting an array A[0..n-2] has already been solved to give us a sorted array of size n-1. Then an appropriate position for A[n-1] is found among the sorted element and then the element is inserted.

42. Give the algorithm for insertion sort.

```

//Sorts a given array by insertion sort
//Input: An array A[0..n-1] of n orderable elements
//Output: Array A[0..n-1] sorted in non-decreasing order
    for I ← 1 to n-1 do
        v ← A[I]
        j ← I-1
        while j >= 0 and A[j] > v do
            A[j+1] ← A[j]
            j ← j - 1
        A[j+1] ← v

```

43. What is time complexity?(Nov'16)

The time complexity of an algorithm is the amount of computer time it needs to run to completion.

44. What is space complexity?

The space complexity of an algorithm is the amount of memory it needs to run to completion.

45. Discuss about various asymptotic notation (UQ Nov'12)

Big oh

The function $f(n) = O(g(n))$ iff there exist positive constants C and n_0 such that $f(n) \leq C * g(n)$ for all $n, n \geq n_0$.

Omega

The function $f(n) = \Omega(g(n))$ iff there exist positive constant C and n_0 such that $f(n) \geq C * g(n)$ for all $n, n \geq n_0$.

Theta

The function $f(n) = \Theta(g(n))$ iff there exist positive constant C_1, C_2 , and n_0 such that $C_1 g(n) \leq f(n) \leq C_2 g(n)$ for all $n, n \geq n_0$.

Little oh

The function $f(n) = o(g(n))$

iff

$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$

$n \rightarrow \infty$

Little Omega.

The function $f(n) = \omega(g(n))$

iff

$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$

$n \rightarrow \infty$

46. What is recursive algorithm? (UQ Nov'12 , Apr/May'14)

An algorithm is said to be recursive if the same algorithm is invoked in the body. An algorithm that calls itself is direct recursive. Algorithm A is said to be indeed recursive if it calls another algorithm, which in turn calls A.

47. Define sorting (UQ Nov'12 , Apr/May'14)

Arrange given number in an order may be ascending or descending order

11 Marks

1. Explain about analysis of algorithms & analysis of control structures. (UQ APRIL'13) (11)

Informal Definition:

An Algorithm is any well-defined computational procedure that takes some value or set of values as Input and produces a set of values or some value as output. Thus algorithm is a sequence of computational steps that transforms the input into the output.

Formal Definition:

An Algorithm is a finite set of instructions that, if followed, accomplishes a particular task. In addition, all algorithms should satisfy the following criteria.

1. INPUT → Zero or more quantities are externally supplied.
2. OUTPUT → At least one quantity is produced.
3. DEFINITENESS → Each instruction is clear and unambiguous.

4. FINITENESS → If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.
5. EFFECTIVENESS → Every instruction must very basic so that it can be carried out, in principle, by a person using only pencil & paper.

Issues or study of Algorithm:

- How to device or design an algorithm → creating and algorithm.
- How to express an algorithm → definiteness.
- How to analysis an algorithm → time and space complexity.
- How to validate an algorithm → fitness.
- Testing the algorithm → checking for error.

Algorithm Specification:

Algorithm can be described in three ways.

1. Natural language like English:
When this way is chosen care should be taken, we should ensure that each & every statement is definite.
2. Graphic representation called flowchart:
This method will work well when the algorithm is small& simple.
3. Pseudo-code Method:
In this method, we should typically describe algorithms as program, which resembles language like Pascal & algol.

Pseudo-Code Conventions:

1. Comments begin with // and continue until the end of line.
2. Blocks are indicated with matching braces {and}.
3. An identifier begins with a letter. The data types of variables are not explicitly declared.

4. Compound data types can be formed with records. Here is an example,

```
Node. Record
{
    data type – 1  data-1;
    .
    .
    .
    data type – n  data – n;
    node * link;
}
```

Here link is a pointer to the record type node. Individual data items of a record can be accessed with → and period.

5. Assignment of values to variables is done using the assignment statement.
<Variable>:= <expression>;

6. There are two Boolean values TRUE and FALSE.

→ Logical Operators AND, OR, NOT
→ Relational Operators <, <=, >, >=, =, !=

7. The following looping statements are employed.

For, while and repeat-until

While Loop:

```
While < condition > do
{
    <statement-1>
    .
    .
    .
    <statement-n>
}
```

For Loop:

For variable: = value-1 to value-2 step step do

```
{
    <statement-1>
    .
    .
    .
    <statement-n>
}
```

repeat-until:

```
repeat
    <statement-1>
    .
    .
    .
    <statement-n>
until <condition>
```

8. A conditional statement has the following forms.

→ If <condition> then <statement>
→ If <condition> then <statement-1>
 Else <statement-1>

Case statement:

```
Case
{
```

```

: <condition-1> : <statement-1>
.
.
.
: <condition-n> : <statement-n>
: else : <statement-n+1>
}

```

9. Input and output are done using the instructions read & write.

10. There is only one type of procedure:
Algorithm, the heading takes the form,

Algorithm Name (Parameter lists)

→ As an example, the following algorithm finds & returns the maximum of 'n' given numbers:

```

1. algorithm Max(A,n)
2. // A is an array of size n
3. {
4.   Result := A[1];
5.   for I:= 2 to n do
6.     if A[I] > Result then
7.       Result :=A[I];
8.   return Result;
9. }

```

In this algorithm (named Max), A & n are procedure parameters. Result & I are Local variables.

→ Next we present 2 examples to illustrate the process of translation problem into an algorithm.

Performance Analysis:

1. Space Complexity:

The space complexity of an algorithm is the amount of memory it needs to run to compilation.

2. Time Complexity:

The time complexity of an algorithm is the amount of computer time it needs to run to compilation.

Space Complexity:

Space Complexity Example:

```

Algorithm abc(a,b,c)
{

```

```

    return a+b++*c+(a+b-c)/(a+b) +4.0;
}

```

→ The Space needed by each of these algorithms is seen to be the sum of the following component.

1. A fixed part that is independent of the characteristics (eg: number, size) of the inputs and outputs.

The part typically includes the instruction space (ie. Space for the code), space for simple variable and fixed-size component variables (also called aggregate) space for constants, and so on.

1. A variable part that consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by referenced variables (to the extent that it depends on instance characteristics), and the recursion stack space.

- The space requirement $s(p)$ of any algorithm p may therefore be written as,

$$S(P) = c + S_p(\text{Instance characteristics})$$

Where 'c' is a constant.

Example 2:

```

Algorithm sum(a,n)
{
    s=0.0;
    for I=1 to n do
        s= s+a[I];
    return s;
}

```

- The problem instances for this algorithm are characterized by n , the number of elements to be summed. The space needed by 'n' is one word, since it is of type integer.
- The space needed by 'a' is the space needed by variables of type array of floating point numbers.
- This is at least 'n' words, since 'a' must be large enough to hold the 'n' elements to be summed.
- So, we obtain $S_{\text{sum}}(n) \geq (n+s)$
[n for a[], one each for n, I a& s]

Time Complexity:

The time $T(p)$ taken by a program P is the sum of the compile time and the run time(execution time)

→ The compile time does not depend on the instance characteristics. Also we may assume that a compiled program will be run several times without recompilation. This run time is denoted by $t_p(\text{instance characteristics})$.

→ The number of steps any problem statement t is assigned depends on the kind of statement.

For example, comments → 0 steps.

Assignment statements → 1 steps.

[Which does not involve any calls to other algorithms]

Interactive statement such as for, while & repeat-until → Control part of the statement.

1. We introduce a variable, count into the program statement to increment count with initial value 0. Statement to increment count by the appropriate amount are introduced into the program.
This is done so that each time a statement in the original program is executed count is incremented by the step count of that statement.

Algorithm:

Algorithm sum(a,n)

```
{
    s= 0.0;
    count = count+1;
    for I=1 to n do
    {
        count =count+1;
        s=s+a[I];
        count=count+1;
    }
    count=count+1;
    count=count+1;
    return s;
}
```

→ If the count is zero to start with, then it will be $2n+3$ on termination. So each invocation of sum execute a total of $2n+3$ steps.

2. The second method to determine the step count of an algorithm is to build a table in which we list the total number of steps contributed by each statement.

→ First determine the number of steps per execution (s/e) of the statement and the total number of times (ie., frequency) each statement is executed.

→ By combining these two quantities, the total contribution of all statements, the step count for the entire algorithm is obtained.

Statement	S/e	Frequency	Total
1. Algorithm Sum(a,n)	0	-	0
2. {	0	-	0
3. S=0.0;	1	1	1
4. for I=1 to n do	1	$n+1$	$n+1$
5. s=s+a[I];	1	n	n
6. return s;	1	1	1
7. }	0	-	0
Total			$2n+3$

AVERAGE –CASE ANALYSIS

- Most of the time, average-case analysis are performed under the more or less realistic assumption that all instances of any given size are equally likely.

- For sorting problems, it is simple to assume also that all the elements to be sorted are distinct.
- Suppose we have 'n' distinct elements to sort by insertion and all n! Permutation of these elements is equally likely.
- To determine the time taken on an average by the algorithm, we could add the times required to sort each of the possible permutations, and then divide by n! The answer thus obtained.
- An alternative approach, easier in this case is to analyze directly the time required by the algorithm, reasoning probabilistically as we proceed.
- For any $1 \leq I \leq n$, consider the sub array, $T[1 \dots i]$.
- The partial rank of $T[I]$ is defined as the position it would occupy if the sub array were sorted.
- For Example, the partial rank of $T[4]$ in $[3,6,2,5,1,7,4]$ is 3 because $T[1 \dots 4]$ once sorted is $[2,3,5,6]$.
- Clearly the partial rank of $T[I]$ does not depend on the order of the element in
- Sub array $T[1 \dots I-1]$.

Analysis

Best case:

This analysis constrains on the input, other than size. Resulting in the fastest possible run time

Worst case:

This analysis constrains on the input, other than size. Resulting in the fastest possible run time

Average case:

This type of analysis results in average running time over every type of input.

Complexity:

Complexity refers to the rate at which the storage time grows as a function of the problem size

Asymptotic analysis:

Expressing the complexity in terms of its relationship to known functions. This type of analysis is called asymptotic analysis.

Asymptotic notation:

Big 'oh': the function $f(n) = O(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n, n \geq n_0$.

Omega: the function $f(n) = \Omega(g(n))$ iff there exist positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n, n \geq n_0$.

Theta: the function $f(n) = \Theta(g(n))$ iff there exist positive constants c_1, c_2 and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n, n \geq n_0$.

2. Write about solving recurrences or Discuss about recursive and non recursive algorithms. (UQ APRIL 12) (UQ: Apr/May '14/ APRIL 2015) (11)

- The indispensable last step when analyzing an algorithm is often to solve a recurrence equation.
- With a little experience and intuition, most recurrence can be solved by intelligent guesswork.
- However, there exists a powerful technique that can be used to solve certain classes of recurrence almost automatically.
- This is a main topic of this section the technique of the characteristic equation.

1. Intelligent guess work:

This approach generally proceeds in 4 stages.

1. Calculate the first few values of the recurrence

2. Look for regularity.
3. Guess a suitable general form.
4. And finally prove by mathematical induction(perhaps constructive induction).

Then this form is correct.

Consider the following recurrence,

$$T(n) = \begin{cases} 0 & \text{if } n=0 \\ 3T(n \div 2) + n & \text{otherwise} \end{cases}$$

- First step is to replace $n \div 2$ by $n/2$
- It is tempting to restrict 'n' to being even since in that case $n \div 2 = n/2$, but recursively dividing an even no. by 2, may produce an odd no. larger than 1.
- Therefore, it is a better idea to restrict 'n' to being an exact power of 2.
- First, we tabulate the value of the recurrence on the first few powers of 2.

n	1	2	4	8	16	32
T(n)	1	5	19	65	211	665

* For instance, $T(16) = 3 * T(8) + 16$
 $= 3 * 65 + 16$
 $= 211.$

* Instead of writing $T(2) = 5$, it is more
 Useful to write $T(2) = 3 * 1 + 2.$

Then,

$$\begin{aligned} T(4) &= 3 * T(2) + 4 \\ &= 3 * (3 * 1 + 2) + 4 \\ &= (3^2 * 1) + (3 * 2) + 4 \end{aligned}$$

* We continue in this way, writing 'n' as an explicit power of 2.

n	T(n)
1	1
2	$3 * 1 + 2$
2^2	$3^2 * 1 + 3 * 2 + 2^2$
2^3	$3^3 * 1 + 3^2 * 2 + 3 * 2^2 + 2^3$
2^4	$3^4 * 1 + 3^3 * 2 + 3^2 * 2^2 + 3 * 2^3 + 2^4$
2^5	$3^5 * 1 + 3^4 * 2 + 3^3 * 2^2 + 3^2 * 2^3 + 3 * 2^4 + 2^5$

- The pattern is now obvious.

$$T(2^k) = 3^k 2^0 + 3^{k-1} 2^1 + 3^{k-2} 2^2 + \dots + 3^1 2^{k-1} + 3^0 2^k.$$

$$\begin{aligned}
&= \sum 3^{k-i} 2^i \\
&= 3^k \sum (2/3)^i \\
&= 3^k * [(1 - (2/3)^{k+1}) / (1 - (2/3))] \\
&= 3^{k+1} - 2^{k+1}
\end{aligned}$$

Proposition: (Geometric Series)

Let S_n be the sum of the first n terms of the geometric series $a, ar, ar^2, \dots$. Then
 $S_n = a(1-r^n)/(1-r)$, except in the special case when $r = 1$; when $S_n = a_n$.

$$\begin{aligned}
&= 3^k * [(1 - (2/3)^{k+1}) / (1 - (2/3))] \\
&= 3^k * [(3^{k+1} - 2^{k+1}) / 3^{k+1}] / ((3 - 2) / 3) \\
&= 3^k * \frac{3^{k+1} - 2^{k+1}}{3^{k+1}} * \frac{3}{1} \\
&= 3^k * \frac{3^{k+1} - 2^{k+1}}{3^{k+1-1}} \\
&= 3^{k+1} - 2^{k+1}
\end{aligned}$$

* It is easy to check this formula against our earlier tabulation.

Eg : 2

$$t_n = \begin{cases} 0 & n=0 \\ 5 & n=1 \\ 3t_{n-1} + 4t_{n-2}, & \text{otherwise} \end{cases}$$

$$t_n - 3t_{n-1} - 4t_{n-2} = 0 \rightarrow \text{General function}$$

$$\begin{aligned}
&\text{Characteristics Polynomial, } x^2 - 3x - 4 = 0 \\
&(x - 4)(x + 1) = 0
\end{aligned}$$

$$\text{Roots } r_1 = 4, r_2 = -1$$

$$\text{General Solution, } f_n = C_1 r_1^n + C_2 r_2^n \rightarrow (A)$$

$$n=0 \rightarrow C_1 + C_2 = 0 \rightarrow (1)$$

$$n=1 \rightarrow C_1 r_1 + C_2 r_2 = 5 \rightarrow (2)$$

$$\text{Eqn 1} \rightarrow C_1 = -C_2$$

sub C_1 value in Eqn (2)

$$-C_2 r_1 + C_2 r_2 = 5$$

$$C_2(r_2 - r_1) = 5$$

$$5$$

$$C_2 = \frac{5}{r_2 - r_1}$$

$$5$$

$$= \frac{-1 + 4}{-1 + 4}$$

$$= 5 / (-5) = -1$$

$$C_2 = -1, C_1 = 1$$

Sub C_1, C_2, r_1 & r_2 value in equation $\rightarrow$ (A)

$$f_n = 1 \cdot 4^n + (-1) \cdot (-1)^n$$

$$f_n = 4^n + 1^n$$

2. Homogenous Recurrences :

* We begin our study of the technique of the characteristic equation with the resolution of homogenous linear recurrences with constant co-efficient, i.e the recurrences of the form,

$$a_0 t_n + a_1 t_{n-1} + \dots + a_k t_{n-k} = 0$$

Where the t_i is the values we are looking for.

* The values of t_i on 'K' values of i (Usually $0 \leq i \leq k-1$ (or) $0 \leq i \leq k$) are needed to determine the sequence.

* The initial condition will be considered later.

* The equation typically has infinitely many solutions.

* The recurrence is,

$\rightarrow$ Linear because it does not contain terms of the form $t_{n-i}, t_{n-j}, t_{n-i}^2$, and so on.

$\rightarrow$ Homogeneous because the linear combination of the t_{n-i} is equal to zero.

$\rightarrow$ With constant co-efficient because the a_i are constants

* Consider for instance our non familiar recurrence for the Fibonacci sequence,

$$f_n = f_{n-1} + f_{n-2}$$

* This recurrence easily fits the mould of equation after obvious rewriting.

$$f_n - f_{n-1} - f_{n-2} = 0$$

* Therefore, the fibonacci sequence corresponds to a homogenous linear recurrence with constant co-efficient with $k=2, a_0=1$ & $a_1=a_2 = -1$.

* In other words, if f_n & g_n satisfy equation.

$$\text{So } \sum_{i=0}^k a_i f_{n-i} = 0 \text{ \& similarly for } g_n \text{ \& } f_n$$

We set $t_n = C f_n + d g_n$ for arbitrary constants C & d , then t_n is also a solution to equation.

* This is true because,

$$\begin{aligned}
 & a_0 t_n + a_1 t_{n-1} + \dots + a_k t_{n-k} \\
 &= a_0(C f_n + d g_n) + a_1(C f_{n-1} + d g_{n-1}) + \dots + a_k(C f_{n-k} + d g_{n-k}) \\
 &= C(a_0 f_n + a_1 f_{n-1} + \dots + a_k f_{n-k}) + \\
 &\quad d(a_0 g_n + a_1 g_{n-1} + \dots + a_k g_{n-k}) \\
 &= C * 0 + d * 0 \\
 &= 0.
 \end{aligned}$$

1) (Fibonacci) Consider the recurrence.

$$f_n = \begin{cases} n & \text{if } n=0 \text{ or } n=1 \\ f_{n-1} + f_{n-2} & \text{otherwise} \end{cases}$$

We rewrite the recurrence as,

$$f_n - f_{n-1} - f_{n-2} = 0.$$

The characteristic polynomial is,

$$x^2 - x - 1 = 0.$$

The roots are,

$$x = \frac{-(-1) \pm \sqrt{((-1)^2 + 4)}}{2}$$

$$= \frac{1 \pm \sqrt{1 + 4}}{2}$$

$$= \frac{1 \pm \sqrt{5}}{2}$$

$$r_1 = \frac{1 + \sqrt{5}}{2} \quad \text{and} \quad r_2 = \frac{1 - \sqrt{5}}{2}$$

The general solution is,

$$f_n = C_1 r_1^n + C_2 r_2^n$$

When $n=0$, $f_0 = C_1 + C_2 = 0$

When $n=1$, $f_1 = C_1 r_1 + C_2 r_2 = 1$

$$C_1 + C_2 = 0 \quad \rightarrow (1)$$

$$C_1 r_1 + C_2 r_2 = 1 \quad \rightarrow (2)$$

From equation (1)

$$C_1 = -C_2$$

Substitute C_1 in equation (2)

$$-C_2r_1 + C_2r_2 = 1$$

$$C_2[r_2 - r_1] = 1$$

Substitute r_1 and r_2 values

$$C_2 \frac{1 - \sqrt{5}}{2} - \frac{1 - \sqrt{5}}{2} = 1$$

$$C_2 \frac{1 - \sqrt{5} - 1 + \sqrt{5}}{2} = 1$$

$$\frac{-C_2 * 2\sqrt{5}}{2} = 1$$

$$-\sqrt{5}C_2 = 1$$

$$C_1 = 1/\sqrt{5}$$

$$C_2 = -1/\sqrt{5}$$

Thus,

$$f_n = \frac{1}{\sqrt{5}} \frac{1 + \sqrt{5}}{2}^n + \frac{-1}{\sqrt{5}} \frac{1 - \sqrt{5}}{2}^n$$

$$= \frac{1}{\sqrt{5}} \frac{1 + \sqrt{5}}{2}^n - \frac{1 - \sqrt{5}}{2}^n$$

3. Inhomogeneous recurrence:

* The solution of a linear recurrence with constant co-efficient becomes more difficult when the recurrence is not homogeneous, that is when the linear combination is not equal to zero.

* Consider the following recurrence

$$a_0t_n + a_1t_{n-1} + \dots + a_k t_{n-k} = b^n p(n)$$

* The left hand side is the same as before,(homogeneous) but on the right-hand side

we have $b^n p(n)$, where,

→ b is a constant

→ $p(n)$ is a polynomial in ' n ' of degree ' d '.

Example (1) :

Consider the recurrence,

$$t_n - 2t_{n-1} = 3^n \rightarrow (A)$$

In this case, $b=3$, $p(n) = 1$, degree = 0.

The characteristic polynomial is,

$$(x - 2)(x - 3) = 0$$

The roots are, $r_1 = 2, r_2 = 3$

The general solution,

$$t_n = C_1 r_1^n + C_2 r_2^n$$

$$t_n = C_1 2^n + C_2 3^n \rightarrow (1)$$

When $n=0$, $C_1 + C_2 = t_0 \rightarrow (2)$

When $n=1$, $2C_1 + 3C_2 = t_1 \rightarrow (3)$

Sub $n=1$ in eqn (A)

$$t_1 - 2t_0 = 3$$

$$t_1 = 3 + 2t_0$$

Substitute t_1 in eqn(3),

$$(2) * 2 \rightarrow \begin{array}{l} 2C_1 + 2C_2 = 2t_0 \\ 2C_1 + 3C_2 = (3 + 2t_0) \end{array}$$

$$-C_2 = -3$$

$$C_2 = 3$$

Sub $C_2 = 3$ in eqn (2)

$$C_1 + C_2 = t_0$$

$$C_1 + 3 = t_0$$

$$C_1 = t_0 - 3$$

$$\begin{aligned} \text{Therefore } t_n &= (t_0 - 3)2^n + 3 \cdot 3^n \\ &= \text{Max}[O[(t_0 - 3) 2^n], O[3 \cdot 3^n]] \\ &= \text{Max}[O(2^n), O(3^n)] \text{ constants} \\ &= O[3^n] \end{aligned}$$

Example :(2)

$$t_n - 2t_{n-1} = (n + 5)3^n, n \geq 1 \rightarrow (A)$$

This is Inhomogeneous

In this case, $b=3$, $p(n) = n+5$, degree = 1

So, the characteristic polynomial is,
 $(x-2)(x-3)^2 = 0$

The roots are,

$$r_1 = 2, r_2 = 3, r_3 = 3$$

The general equation,

$$t_n = C_1 r_1^n + C_2 r_2^n + C_3 n r_3^n \rightarrow (1)$$

When $n=0$, $t_0 = C_1 + C_2 \rightarrow (2)$

When $n=1$, $t_1 = 2C_1 + 3C_2 + 3C_3 \rightarrow (3)$

Substituting $n=1$ in eqn(A),

$$t_1 - 2t_0 = 6 \cdot 3$$

$$t_1 - 2t_0 = 18$$

$$t_1 = 18 + 2t_0$$

Substituting t_1 value in eqn(3)

$$2C_1 + 3C_2 + 3C_3 = 18 + 2t_0 \rightarrow (4)$$

$$C_1 + C_2 = t_0 \rightarrow (2)$$

Sub. $n=2$ in eqn(1)

$$4C_1 + 9C_2 + 18C_3 = t_2 \rightarrow (5)$$

Sub $n=2$ in eqn (A)

$$t_2 - 2t_1 = 7 \cdot 9$$

$$t_2 = 63 + 2t_1$$

$$= 63 + 2[18 + 2t_0]$$

$$t_2 = 63 + 36 + 4t_0$$

$$t_2 = 99 + 4t_0$$

Sub. t_2 value in eqn(3),

$$4C_1 + 9C_2 + 18C_3 = 99 + 4t_0 \rightarrow (5)$$

Solve eqn (2),(4) & (5)

$$n=0, C_1 + C_2 = t_0 \rightarrow (2)$$

$$n=1, 2C_1 + 3C_2 + 3C_3 = 18 + 2t_0 \rightarrow (4)$$

$$n=2, 4C_1 + 9C_2 + 18C_3 = 99 + 4t_0 \rightarrow (5)$$

$$(4) * 6 \rightarrow 12C_1 + 18C_2 + 18C_3 = 108 + 2t_0 \rightarrow (4)$$

$$(5) \rightarrow 4C_1 + 9C_2 + 18C_3 = 99 + 4t_0 \rightarrow (5)$$

$$8C_1 + 9C_2 = 9 + 8t_0 \rightarrow (6)$$

$$(2) * 8 \rightarrow 8C_1 + 8C_2 = 8t_0 \rightarrow (2)$$

$$(6) \rightarrow 8C_1 + 9C_2 = 9 + 8t_0 \rightarrow (6)$$

$$-C_2 = -9$$

$$C_2 = 9$$

Sub, $C_2 = 9$ in eqn(2)

$$C_1 + C_2 = t_0$$

$$C_1 + 9 = t_0$$

$$C_1 = t_0 - 9$$

Sub C_1 & C_2 in eqn (4)

$$2C_1 + 3C_2 + 3C_3 = 18 + 2t_0$$

$$2(t_0 - 9) + 3(9) + 3C_3 = 18 + 2t_0$$

$$2t_0 - 18 + 27 + 3C_3 = 18 + 2t_0$$

$$2t_0 + 9 + 3C_3 = 18 + 2t_0$$

$$3C_3 = 18 - 9 + 2t_0 - 2t_0$$

$$3C_3 = 9$$

$$C_3 = 9/3$$

$$C_3 = 3$$

Sub. $C_1, C_2, C_3, r_1, r_2, r_3$ values in eqn (1)

$$\begin{aligned} t_n &= C_1 2^n + C_2 3^n + C_3 \cdot n \cdot 3^n \\ &= (t_0 - 9) 2^n + 9 \cdot 3^n + 3 \cdot n \cdot 3^n \\ &= \text{Max}[O[(t_0 - 9) \cdot 2^n], O[9 \cdot 3^n], O[3 \cdot n \cdot 3^n]] \end{aligned}$$

$$= \text{Max}[O(2^n), O(3^n), O(n \cdot 3^n)]$$

$$t_n = O[n \cdot 3^n]$$

Example: (3)

Consider the recurrence,

$$t_n = \begin{cases} 1 & \text{if } n=0 \\ 4t_{n-1} - 2^n & \text{otherwise} \end{cases}$$

$$t_n - 4t_{n-1} = -2^n \rightarrow (A)$$

In this case, $c=2, p(n) = -1, \text{degree} = 0$

$$(x-4)(x-2) = 0$$

The roots are, $r_1 = 4, r_2 = 2$

The general solution,

$$t_n = C_1 r_1^n + C_2 r_2^n$$

$$t_n = C_1 4^n + C_2 2^n \rightarrow (1)$$

$$\text{when } n=0, \text{ in (1)} \rightarrow C_1 + C_2 = 1 \rightarrow (2)$$

$$\text{when } n=1, \text{ in (1)} \rightarrow 4C_1 + 2C_2 = t_1 \rightarrow (3)$$

sub $n=1$ in (A),

$$t_n - 4t_{n-1} = -2^n$$

$$t_1 - 4t_0 = -2$$

$$t_1 = 4t_0 - 2 \quad [\text{since } t_0 = 1]$$

$$t_1 = 2$$

Sub t_1 value in eqn (3)

$$4C_1 + 2C_2 = 4t_0 - 2 \rightarrow (3)$$

$$(2) * 4 \rightarrow 4C_1 + 4C_2 = 4$$

$$\begin{array}{r} -2C_2 = 4t_0 - 6 \\ \quad = 4(1) - 6 \\ \quad = -2 \\ C_2 = 1 \end{array}$$

$$-2C_2 = 4t_0 - 6$$

$$2C_2 = 6 - 4t_0$$

$$C_2 = 3 - 2t_0$$

$$3 - 2(1) = 1$$

$$C_2 = 1$$

Sub. C_2 value in eqn(2),

$$C_1 + C_2 = 1$$

$$C_1 + (3-2t_0) = 1$$

$$C_1 + 3 - 2t_0 = 1$$

$$C_1 = 1 - 3 + 2t_0$$

$$C_1 = 2t_0 - 2$$

$$= 2(1) - 2 = 0$$

$$C_1 = 0$$

Sub C_1 & C_2 value in eqn (1)

$$t_n = C_1 4^n + C_2 2^n$$

$$= \text{Max}[O(2t_0 - 2).4^n, O(3 - 2t_0).2^n]$$

$$= \text{Max}[O(2^n)]$$

$$t_n = O(2^n)$$

Example: (4)

$$t_n = \begin{cases} 0 & \text{if } n=0 \\ 2t_{n-1} + n + 2^n & \text{otherwise} \end{cases}$$

$$t_n - 2t_{n-1} = n + 2^n \rightarrow (A)$$

There are two polynomials.

For n ; $b=1$, $p(n)$, degree = 1

For $2n$; $b=2$, $p(n) = 1$, degree = 0

The characteristic polynomial is,

$$(x-2)(x-1)^2(x-2) = 0$$

The roots are, $r_1 = 2$, $r_2 = 2$, $r_3 = 1$, $r_4 = 1$.

So, the general solution,

$$t_n = C_1 r_1^n + C_2 n r_2^n + C_3 r_3^n + C_4 n r_4^n$$

Sub r_1, r_2, r_3 in the above eqn

$$t_n = 2^n C_1 + 2^n C_2 n + C_3 \cdot 1^n + C_4 \cdot n \cdot 1^n \rightarrow (1)$$

$$\text{Sub. } n=0 \rightarrow C_1 + C_3 = 0 \rightarrow (2)$$

$$\text{Sub. } n=1 \rightarrow 2C_1 + 2C_2 + C_3 + C_4 = t_1 \rightarrow (3)$$

Sub. $n=1$ in eqn (A)

$$t_n - 2t_{n-1} = n + 2^n$$

$$t_1 - 2t_0 = 1 + 2$$

$$t_1 - 2t_0 = 3$$

$$t_1 = 3 \quad [\text{since } t_0 = 0]$$

Sub. $n=2$ in eqn (1)

$$2^2 C_1 + 2 \cdot 2^2 \cdot C_2 + C_3 + 2 \cdot C_4 = t_2$$

$$4C_1 + 8C_2 + C_3 + 2C_4 = t_2$$

Sub $n=2$ in eqn (A)

$$t_2 - 2t_1 = 2 + 2^2$$

$$t_2 - 2t_1 = 2 + 4$$

$$t_2 - 2t_1 = 6$$

$$t_2 = 6 + 2t_1$$

$$t_2 = 6 + 2.3$$

$$t_2 = 6 + 6$$

$$t_2 = 12$$

$$\rightarrow 4C_1 + 8C_2 + C_3 + 2C_4 = 12 \rightarrow (4)$$

Sub $n=3$ in eqn (!)

$$2^3 C_1 + 3.2^3.C_2 + C_3 + 3C_4 = t_3$$

$$3C_1 + 24C_2 + C_3 + 3C_4 = t_3$$

Sub $n=3$ in eqn (A)

$$t_3 - 2t_2 = 3 + 2^3$$

$$t_3 - 2t_2 = 3 + 8$$

$$t_3 - 2(12) = 11$$

$$t_3 - 24 = 11$$

$$t_3 = 11 + 24$$

$$t_3 = 35$$

$$\rightarrow 8C_1 + 24C_2 + C_3 + 3C_4 = 35 \rightarrow (5)$$

$$n=0, \text{ solve; } C_1 + C_3 = 0 \rightarrow (2)$$

$$n=1, (2), (3), (4) \& (5) \quad 2C_1 + 2C_2 + C_3 + C_4 = 3 \rightarrow (3)$$

$$n=2, \quad 4C_1 + 8C_2 + C_3 + 2C_4 = 12 \rightarrow (4)$$

$$n=3, \quad 8C_1 + 24C_2 + C_3 + 3C_4 = 35 \rightarrow (5)$$

$$-4C_1 - 16C_2 - C_4 = -23 \rightarrow (6)$$

Solve: (2) & (3)

$$(2) \rightarrow C_1 + C_3 = 0$$

$$(3) \rightarrow 2C_1 + C_3 + 2C_2 + C_4 = 3$$

$$-C_1 - 2C_2 - C_4 = -3 \rightarrow (7)$$

Solve (6) & (7)

$$(6) \rightarrow -4C_1 - 16C_2 - C_4 = -23$$

$$(7) \rightarrow -C_1 - 2C_2 - C_4 = -3$$

$$-3C_1 - 14C_2 = 20 \rightarrow (8)$$

4. Change of variables:

* It is sometimes possible to solve more complicated recurrences by making a change of variable.

* In the following example, we write $T(n)$ for the term of a general recurrences, and t_i for the term of a new recurrence obtained from the first by a change of variable.

Example: (1)

Consider the recurrence,

$$T(n) = \begin{cases} 1 & , \text{ if } n=1 \\ 3T(n/2) + n & , \text{ if 'n' is a power of 2, } n>1 \end{cases}$$

→ Reconsider the recurrence we solved by intelligent guesswork, but only for the case when 'n' is a power of 2

$$T(n) = \begin{cases} 1 \\ 3T(n/2) + n \end{cases}$$

* We replace 'n' by 2^i .

* This is achieved by introducing new recurrence t_i , define by $t_i = T(2^i)$

* This transformation is useful because $n/2$ becomes $(2^i)/2 = 2^{i-1}$

* In other words, our original recurrence in which $T(n)$ is defined as a function of $T(n/2)$ given way to one in which t_i is defined as a function of t_{i-1} , precisely the type of recurrence we have learned to solve.

$$t_i = T(2^i) = 3T(2^{i-1}) + 2^i$$

$$t_i = 3t_{i-1} + 2^i$$

$$t_i - 3t_{i-1} = 2^i \rightarrow (A)$$

In this case,

$$b = 2, p(n) = 1, \text{ degree} = 0$$

So, the characteristic equation,

$$(x - 3)(x - 2) = 0$$

The roots are, $r_1 = 3, r_2 = 2$.

The general equation,

$$t_n = C_1 r_1^i + C_2 r_2^i$$

Sub. r_1 & r_2 : $t_n = 3^n C_1 + C_2 2^n$

$$t_n = C_1 3^i + C_2 2^i$$

We use the fact that, $T(2^i) = t_i$ & thus $T(n) = t_{\log n}$ when $n = 2^i$ to obtain,

$$T(n) = C_1 \cdot 3^{\log_2 n} + C_2 \cdot 2^{\log_2 n}$$

$$T(n) = C_1 \cdot n^{\log_2 3} + C_2 \cdot n \quad [i = \log n]$$

When 'n' is a power of 2, which is sufficient to conclude that,

$$T(n) = O(n^{\log_2 3}) \text{ 'n' is a power of 2}$$

Example: (2)

Consider the recurrence,

$$T(n) = 4T(n/2) + n^2 \rightarrow (A)$$

Where 'n' is a power of 2, $n \geq 2$.

$$t_i = T(2^i) = 4T(2^{i-1}) + (2^i)^2$$

$$t_i = 4t_{i-1} + 4^i$$

$$\rightarrow t_i - 4t_{i-1} = 4^i$$

In this eqn,

$$b = 4, P(n) = 1, \text{degree} = 0$$

The characteristic polynomial,

$$(x - 4)(x - 4) = 0$$

The roots are, $r_1 = 4, r_2 = 4$.

So, the general equation,

$$\begin{aligned} t_i &= C_1 4^i + C_2 4^i \cdot i && [\text{since } i = \log n] \\ &= C_1 4^{\log n} + C_2 \cdot 4^{\log n} \cdot \log n && [\text{since } 2^i = n] \\ &= C_1 \cdot n^{\log 4} + C_2 \cdot n^{\log 4} \cdot n^{\log 1} \end{aligned}$$

$$T(n) = O(n^{\log 4}) \text{ 'n' is the power of 2.}$$

Example: 3

$$T(n) = 2T(n/2) + n \log n$$

When 'n' is a power of 2, $n \geq 2$

$$t_i = T(2^i) = 2T(2^{i-1}) + 2^i \cdot i \quad [\text{since } 2^i = n; i = \log n]$$

$$t_i - 2t_{i-1} = i \cdot 2^i$$

In this case,

$$b = 2, P(n) = i, \text{degree} = 1$$

$$(x - 2)(x - 2)^2 = 0$$

The roots are, $r_1 = 2, r_2 = 2, r_3 = 2$

The general solution is,

$$\begin{aligned} t_n &= C_1 2^i + C_2 \cdot 2^i \cdot i + C_3 \cdot i^2 \cdot 2^i \\ &= nC_1 + nC_2 + nC_3(\log^2 n) \end{aligned}$$

$$t_n = O(n \cdot \log^2 n)$$

Example: 4

$$T(n) = \begin{cases} 2 & , n=1 \\ 5T(n/4) + Cn^2 & , n>1 \end{cases}$$

$$\begin{aligned} t_i &= T(4^i) = 5T(4^{i-1}) + C(4^i)^2 \\ &= 5T(4^{i-1}) + C \cdot 16^i \\ &= 5t_{i-1} + C \cdot 16^i \end{aligned}$$

$$t_i - 5t_{i-1} = C \cdot 16^i$$

In this case,

$$b = 16, P(n) = 1, \text{degree} = 0$$

The characteristic eqn,

$$(x - 5)(x - 16) = 0$$

The roots are, $r_1 = 5, r_2 = 16$

The general solution,

$$\begin{aligned} t_i &= C_1 \cdot 5^i + C_2 \cdot 16^i \\ &= C_1 \cdot 5^i + C_2 \cdot (4^2)^i \\ t_n &= O(n^2) \end{aligned}$$

Example: 5

$$T(n) = \begin{cases} 2 & , n = 1 \\ T(n/2) + Cn & , n > 1 \end{cases}$$

$$\begin{aligned} T(n) &= T(n/2) + Cn \\ &= T(2^i/2) + C \cdot 2^i \\ &= T(2^{i-1}) + C \cdot 2^i \\ t_i &= t_{i-1} + C \cdot 2^i \\ t_i - t_{i-1} &= C \cdot 2^i \end{aligned}$$

In this case, $b=2, P(n)=1, \text{degree}=0$

So, the characteristic polynomial,

$$(x - 1)(x - 2) = 0$$

The roots are, $r_1 = 1, r_2 = 2$

$$\begin{aligned} t_i &= C_1 \cdot 1^i + C_2 \cdot 2^i \\ &= C_1 \cdot 1^{\log_2 n} + C_2 \cdot n \\ &= C_1 \cdot n^{\log_2 1} + C_2 \cdot n \end{aligned}$$

$$t_n = O(n)$$

Example: 6

$$T(n) = \begin{cases} 1 & , n = 1 \\ 3T(n/2) + n; n \text{ is a power of } 2 \end{cases}$$

$$\begin{aligned} t_i &= T(2^i) = 3T(2^{i-1}) + 2^i \\ &= 3T(2^{i-1}) + 2^i \\ t_i &= 3t_{i-1} + 2^i \end{aligned}$$

So, $b = 2, P(n) = 1, \text{degree} = 0$

$$(x - 3)(x - 2) = 0$$

The roots are, $r_1 = 3, r_2 = 2$

$$\begin{aligned}
 t_i &= C_1 \cdot 3^i + C_2 \cdot 2^i \\
 &= C_1 \cdot 3^{\log_2 n} + C_2 \cdot 2^{\log_2 n} \\
 &= C_1 \cdot n^{\log_2 3} + C_2 \cdot n^{\log_2 2} = 1 \\
 &= C_1 \cdot n^{\log_2 3} + C_2 \cdot n
 \end{aligned}$$

$$t_n = O(n^{\log_2 3})$$

Example: 7

$$T(n) = 2T(n/2) + n \cdot \log n$$

$$\begin{aligned}
 t_i &= T(2^i) = 2T(2^i/2) + 2^i \cdot i \\
 &= 2T(2^{i-1}) + i \cdot 2^i \\
 &= 2t_{i-1} + i \cdot 2^i \\
 t_i - 2t_{i-1} &= i \cdot 2^i
 \end{aligned}$$

→ b=2, P(n) = I, degree = 1

The roots is $(x - 2)(x - 2)^2 = 0$
 $x = 2, 2, 2$

General solution,

$$\begin{aligned}
 t_n &= C_1 \cdot r_1^i + C_2 \cdot i \cdot r_2^i + C_3 \cdot i^2 \cdot r_3^i \\
 &= C_1 \cdot 2^i + C_2 \cdot i \cdot 2^i + C_3 \cdot i^2 \cdot 2^i \\
 &= C_1 \cdot n + C_2 \cdot n \cdot \log_2 n + C_3 \cdot i^2 \cdot n \\
 &= C_1 \cdot n + C_2 \cdot n \cdot \log_2 n + C_3 (2^{\log_2 n}) \cdot n
 \end{aligned}$$

$$t_n = O(n \cdot 2^{\log_2 n})$$

5. Range Transformation:

- * When we make a change of variable, we transform the domain of the recurrence.
- * Instead it may be useful to transform the range to obtain a recurrence in a form that we know how to solve
- * Both transformation can be sometimes be used together.

Example: 1

Consider the following recurrence, which defines $T(n)$. where 'n' is the power of 2

$$T(n) = \begin{cases} 1/3 & , \text{ if } n=1 \\ n T^2(n/2) & , \text{ otherwise} \end{cases}$$

The first step is a change of variable,

$$\begin{aligned}
 \text{Let } t_i &\text{ denote } T(2^i) \\
 t_i &= T(2^i) = 2^i T^2(2^{i-1}) \\
 &= 2^i t_{i-1}^2
 \end{aligned}$$

- * This recurrence is not clear, furthermore the co-efficient 2^i is not a constant.
- * To transform the range, we create another recurrence by using u_i to denote $\lg t_i$

$$u_i = \lg t_i = i + 2 \lg t_{i-1} \\ = i + 2u_{i-1}$$

$$\rightarrow u_i - 2u_{i-1} = i \\ (x - 2)(x - 1)^2 = 0$$

The roots are, $x = 2, 1, 1$.

G.S,

$$u_i = C_1 \cdot 2^i + C_2 \cdot 1^i + C_3 \cdot i \cdot 1^i$$

Sub. This solution into the recurrence,

For u_i yields,

$$i = u_i - 2u_{i-1} \\ = C_1 2^i + C_2 + C_3 \cdot i - 2(C_1 \cdot 2^{i-1} + C_2 + C_3 (i-1)) \\ = (2C_3 - C_2) - C_3 i.$$

$$C_3 = -1 \text{ \& } C_2 = 2C_3 = -2$$

$$u_i = C_1 2^i - i - 2$$

This gives us the G.S for t_i & $T(n)$

$$t_i = 2^{u_i} = 2^{C_1 2^i - i - 2} \\ T(n) = t_{\lg n} = 2^{C_1 n - \lg n - 2} \\ = 2^{C_1 n} / 4n$$

We use the initial condition $T(1) = 1/3$

To determine C_1 : $T(1) = 2^{C_1} / 4 = 1/3$

Implies that $C_1 = \lg(4/3) = 2 - \log 3$

The final solution is

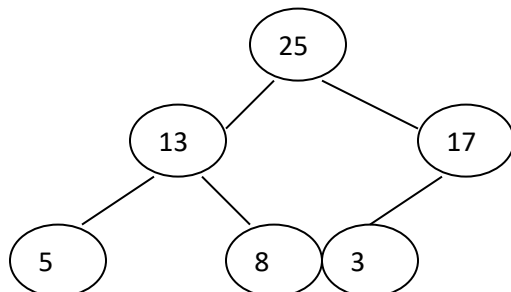
$$T(n) = 2^{2n} / 4n 3^n$$

3. Write and analyze about heap sort (UQ Nov'12)

(11)

A heap is a complete binary tree with the property that the value at each node is at least as large as the value at its children.

The definition of a max heap implies that one of the largest elements is at the root of the heap. If the elements are distinct then the root contains the largest item. The tree is completely filled on all levels except possibly lowest.



An array [25, 13, 17, 5, 8, 3] is represented as heap above. The maximum element in an array is always a root node.

- The indices of its parent, left child and right child can be computed as
 $\text{PARENT}(i) = \text{floor}(i/2)$
 $\text{LEFT}(i) = 2i$
 $\text{RIGHT}(i) = 2i + 1$
- All the tree levels are completely filled except possibly for the lowest level, which is filled from the left up to a point.
- The levels above the lowest level from a complete binary tree of height $h-1$ contain $2^h - 1$ node.
- A heap of height h has
 - Maximum no. of elements $= 2^{h+1}$ nodes. (When lowest level is completely filled)
 - Minimum no. of nodes $= 2^h$ nodes.

Height of a node:

It is defined as no. of edges on a simple downward path from a node to leaf.

Height of a tree:

It is defined as no. of edges on a simple downward path from a root node to leaf.

To insert an element into the heap, one adds it "at the bottom" of the heap and then compares it with its parent, grandparent, great grandparent and so on, until it is less than or equal to one of these values.

Example:

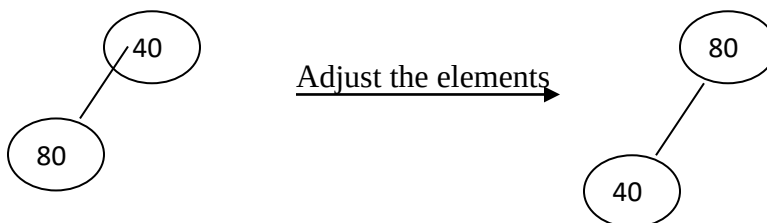
a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
40	80	35	90	45	50	70

Step1: Take the first element of an array. $a[1]=40$.

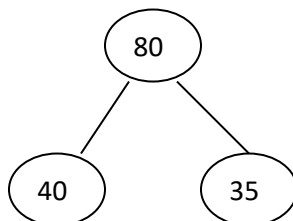


Step 2: Take the second element of an array. $a[2]=80$.

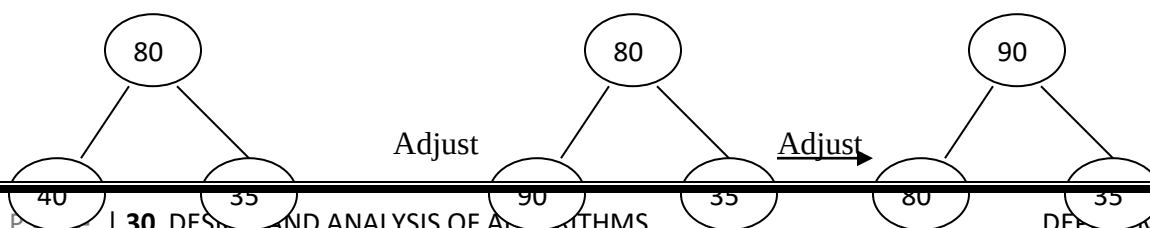
Insert $a[2]$ to the left side of the first element.

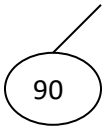


Step 3: Take the 3rd element. $a[3]=35$. Add this element to the right side of the root node.

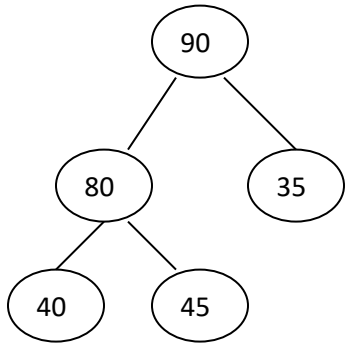


Step 4: Take the 4th element. $a[4]=90$. Add this element to the left side of the node 40.

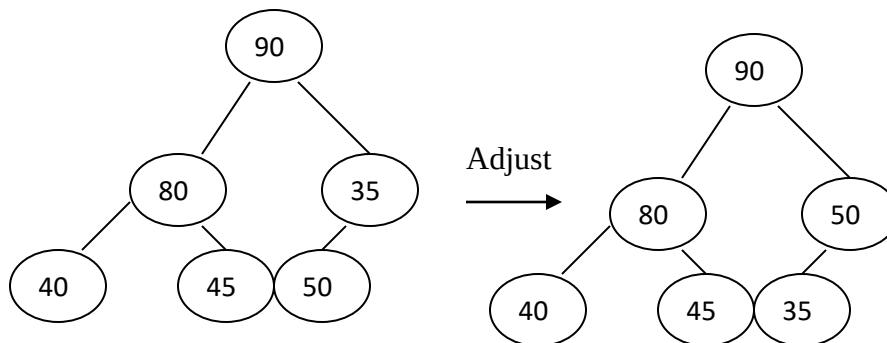




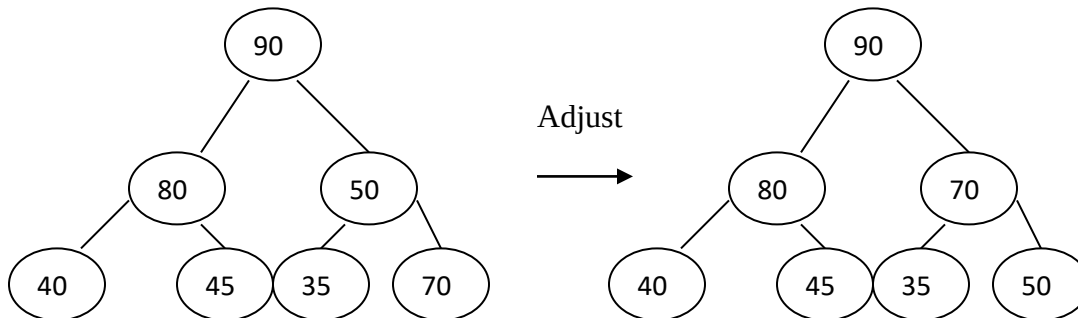
Step 5: Take the 5th element .a[5]=45.Add this element to the right side of the node 80.



Step 6: Take the 6th element .a[6]=50.Add this element to the left side of the node 35.



Step 7: Take the 7th element .a[7]=70.Add this element to the right side of the node 50.



The figure shows one example of how insert () would insert a new value into an existing heap. To do heap sort, first add the first element in the tree. Then add the second element to the tree. If the tree does not satisfy the heap property then adjust the nodes in the tree. Then insert the next element and adjust the nodes.

Repeat the above steps till the insertion of last element. Finally we obtain the sorted elements in heap tree representation.

Procedures for Heap Sort:

1. Build a Heap(Heapify)
2. Adjust

Procedure Heapify():

- Create a heap from n elements.
- Maintain the heap property. That's why we are using adjust () procedure to arrange the elements.

Procedure Adjust ():

Adjust takes as input the array a[] and integer I and n. It regards a[1..n] as a complete binary tree. If the sub trees rooted at 2I and 2I+1 are max heaps, then adjust will rearrange elements of a[] such that the tree rooted at I is also a max heap. The maximum elements from the max heap a[1..n] can be deleted by deleting the root of the corresponding complete binary tree. The last element of the array, i.e. a[n], is copied to the root, and finally we call Adjust(a,1,n-1).

Algorithm Heapsort(a,n)

```
{
    Heapify(a,n);
    //Transform the array into heap.
    //Interchange the new max with the element at the end of the array.
    for i=n to 2 step -1 do
    {
        t:=a[i];
        a[i]:=a[1];
        a[1]:=t;
        Adjust(a,1,i-1);
    }
}
```

Algorithm Heapify(a,n)

```
{
    //Readjust the elements in a[1:n] to form a heap.
    for i:= n/2 to 1 step -1 do
        Adjust(a,i,n);
}
```

Algorithm Adjust(a,i,n)

```
{
    //The complete binary trees with roots 2i and 2i+1 are
    //combined with node i to form a heap rooted at i.
    //No node has an address greater than n or less than 1.
    j=2i;
    item=a[i];
    while (j<=n) do
    {
        if ((j<n) and (a[j]< a[j+1])) then
            j=j+1;
        //compare left and right child and let j be the larger child
        if ( item >= a[j]) then break;
        // A position for item is found
    }
```



```

        a[j/2]=a[j];
        j=2j;
    }
    a[j/2]=item;
}

```

Analysis:

- Heapify() requires n operations. $\rightarrow O(n)$
- Adjust() requires $O(\log n)$ operations for each invocation. Adjust is called n times by Heapsort(). So time is $O(n \log n)$.
- Heapsort = $O(n) + O(n \log n) = O(n \log n)$.

Algorithm HeapSort():
 $T(n) = O(n \log n)$

Algorithm Heapify():
 $T(n) = O(\log n)$

Algorithm Adjust():
 $T(n) = O(\log n)$

4. Write in detail about radix sort (UQ APRIL12/ APRIL 2015) (6)

It is a small method..It is a list of successive sort on increasingly significant digit position. On the first pass the entire numbers are sorted based on the least significant digit and combine in an array. Then on the second pass, the entire numbers are sorted on the least significant digit and combine in an array and so on.

Each element in the n th element array A has d digits, where digit 1 is the lower order digit and d is the highest order digit.

This algorithm is based on properties of keys. All other algorithms are named as comparison sort. But in this algorithm instead of comparing the elements, we sort the elements bit by bit.

Pseudo Code Algorithm:

Algorithm radixsort(a, n, d)

```

{
    for i:= 1 to d do
    {
        for j:=0 to 9 do
        {
            for k:=0 to 9 do
                b[i][k]:=0;
            for j:=1 to n do
            {
                k:=[ a[j] % pow(10,i) - a[j] % pow(10,i-1) ] / pow(10,i-1);
                b[k][0]:=b[k][0]+1;
                b[k][b[k][0]]:=a[j];
            }
        }
    }
}

```

```

    }
  }
}
  for j:=0 to 9 do
  {
    for k:=2 to b[j][1] do
    {
      a[b++]:=b[j][k+1];
    }
  }
}
Algorithm pow(a,b)
{
  temp:=1;
  for i:= 1 to b do
    temp:=temp*a;
  return temp;
}

```

Example:

Input	Pass1	Pass2	Pass3
329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657]
720	329	457	720
355	839	657	839

Analysis:

- Running time depends on the stable sort.
- Each digit is in the range of 1 to k.
- Each pass over n takes $\theta(n+k)$ time.
- There are d no. of passes.
- Total time for radix sort is $\theta(dn+kd)$ where d is constant and k is $\theta(n)$
- The radix sort is linear in time.

5. Write about Insertion sort

(6)

If the first few objects are already sorted, an unsorted object can be inserted in the sorted set in proper place. This is called insertion sort.

An algorithm consider the elements one at a time, inserting each in its suitable place among those already considered (keeping them sorted). Insertion sort is an example of an incremental algorithm; it builds the sorted sequence one number at a time.

Pseudo Code Algorithm:

Algorithm Insertion_Sort(A,n)

1. For j = 2 to length [A] do
2. key = A[j]
3. {Put A[j] into the sorted sequence A[1 . . j-1]}
4. i ← j -1
5. while i > 0 and A[i] > key do
6. A[i+1] = A[i]

7. $i = i-1$
8. $A[i+1] = \text{key}$

Analysis:

Best-Case:

The while-loop in line 5 executed only once for each j . This happens if given array A is already sorted.

$$T(n) = an + b = O(n)$$

$$T(n) = O(n)$$

It is a linear function of n .

Worst-Case:

The worst-case occurs, when line 5 executed j times for each j . This can happen if array A starts out in reverse order.

$$T(n) = an^2 + bn + c = O(n^2)$$

$$T(n) = O(n^2)$$

It is a quadratic function of n .

Extra Memory

This algorithm does not require extra memory.

6. Write and analyze selection sort (UQ APRIL13 & Nov'12) (11)

Selection sort is the simplest of sorting techniques. The idea in selection sort is to find the smallest element in the array and exchange it with the element in the first position, then find the second smallest element and exchange it with the element in the second position, and continue in this way until the entire array is sorted.

Pseudo Code Algorithm:

Algorithm Selection_Sort (A, n)

```
{
    for i := 1 to n-1 do
    {
        min := i;
        m:=a[min];
        for j: = i+1 to n do
        {
            if (a[j] < m) then
                min = j;
        }
        t := a[i];
        a[min] := a[i];
        a[i] = t;
    }
}
```

Example:

Consider the unsorted array, $n=8$

$a[1] \ a[2] \ a[3] \ a[4] \ a[5] \ a[6] \ a[7] \ a[8]$

20	35	18	8	14	41	3	39
----	----	----	---	----	----	---	----

After sorting, the resulting array should be

$a[1] \ a[2] \ a[3] \ a[4] \ a[5] \ a[6] \ a[7] \ a[8]$

3	8	14	18	20	35	39	41
---	---	----	----	----	----	----	----

When i=1

- Find the smallest element in the unsorted array.
- Place the smallest element in position of a[1]
i.e., the smallest element in the unsorted array is 3 so exchange the values of a[1] and a[7].

a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8]

3	35	18	8	14	41	20	39
---	----	----	---	----	----	----	----

When i=2

- Find the smallest from a[2] to a[8] , i.e., 8 in a[4]
- Exchange the values of a[2] and a[4].

a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8]

3	8	18	35	14	41	20	39
---	---	----	----	----	----	----	----

When i=3

- Find the smallest from a[3] to a[8] , i.e., 14 in a[5]
- Exchange the values of a[3] and a[5].

a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8]

3	8	14	35	18	41	20	39
---	---	----	----	----	----	----	----

When i=4

- Find the smallest from a[4] to a[8] , i.e., 18 in a[5]
- Exchange the values of a[4] and a[5].

a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8]

3	8	14	18	35	41	20	39
---	---	----	----	----	----	----	----

When i=5

- Find the smallest from a[5] to a[8] , i.e., 20 in a[7]
- Exchange the values of a[5] and a[7].

a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8]

3	8	14	18	20	41	35	39
---	---	----	----	----	----	----	----

When i=6

- Find the smallest from a[6] to a[8] , i.e., 35 in a[7]
- Exchange the values of a[6] and a[7].

a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8]

3	8	14	18	20	35	41	39
---	---	----	----	----	----	----	----

When i=7

- Find the smallest from a[7] to a[8] , i.e., 39 in a[8]
- Exchange the values of a[7] and a[8].

a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8]

3	8	14	18	20	35	39	41
---	---	----	----	----	----	----	----

The number of moves with this technique is always of the order $O(n)$.

Selection sort works very well for small files. It has a quite important application because each item is actually moved at most once. It is a method of choice for sorting files with very large objects (records) and small keys.

Analysis:

The number of moves with this technique is always of the order $O(n)$. So the time complexity is $O(n)$.

7. Write and analyze about bubble sort (UQ APRIL 13/APRIL 2015) (6)

Bubble Sort is an elementary sorting algorithm. It is an oldest, slowest and simple method. It is a sorting by comparing each adjacent pair of items in a list, swapping the items if necessary and repeats the process thru the list until no swaps are done.

Steps:

1. Compare the first 2 elements.
2. If the first element is greater than second, then swap them.
3. Otherwise, leave as it is.
4. Then 2nd and 3rd pass. Elements are compared and swap them if needed.
5. Thus adjacent 2 elements are compared till all the elements have been passed through once.
6. Thus the largest element is positioned at the last position.
7. Then repeat the steps 2, 3, 4 & 5 are repeated until no swaps have been done on the last pass.
8. At the end of the 2nd pass the 2nd largest element is positioned in the corresponding location.
9. For n data items, the process continue for (n-1) passes or until no exchanges are made in a single

Pseudo Code Algorithm:

Algorithm Bubblesort (A,n)

```
{
    For i:=1 to n do
        for j:=n to i +1 step -1 do
            if ( a[j] < a[j-1] ) then
            {
                t:=a[j] ;
                a[j]:=a[j-1];
                a[j-1]=t;
            }
        }
    }
```

Example:

Analysis:

- The number of comparisons made
 $1 + 2 + 3 + \dots + (n - 1) = n(n - 1)/2 = O(n^2)$
- In this algorithm the number of comparison is irrespective of data set i.e., input whether best or worst.
- Time Complexity $T(n) = O(n^2)$
- It does not require extra memory.

8. Write in detail about searching or Explain about Sequential and Fibonacci search (UQ APRIL13 & Nov'10) (UQ: Apr/May'14) (11)

Let us assume that we have a sequential file and we wish to retrieve an element matching with key 'k', then, we have to search the entire file from the beginning till the end to check whether the element matching k is present in the file or not.

There are a number of complex searching algorithms to serve the purpose of searching. The linear search and binary search methods are relatively straight forward methods of searching.

Sequential search

In this method, we start to search from the beginning of the list and examine each element till the end of the list. If the desired element is found we stop the search and return the index of that element. If the item is not found and the list is exhausted the search returns a zero value.

Pseudo Code Algorithm:

Algorithm sequential(a[],n,x)

```
{
    i:=1;
    while(i<=n)
    {
        if(x==a[i])
        {
            write("Element Found in ",i);
            write ('search successful');
            exit( );
        }
        else
        {
            i:=i+1;
        }
    }
    write ('search unsuccessful');
}
```

Analysis:

Worst Case:

The worst case occurs when an item is not found or the search item is the last (n^{th}) element. For both situations we must examine all n elements of the array. So the complexity of the sequential search is n , i.e., $O(n)$. The execution time is proportional to n , i.e. the algorithm is executed in linear time.

Best Case:

The best case occurs when the search item is at the first location itself. So the time complexity is $O(1)$.

Average Case:

The average case occurs if the search element is at the middle location. So we must examine $n/2$ elements of the array. So the time complexity is $O(n/2)$.

Example:

a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8]

20	35	18	8	14	41	3	39
----	----	----	---	----	----	---	----

Best Case:

If $x=20$ then element is found at the first location.

$T(n)=O(1)$

Average Case:

If $x=8$ then we must compare the 4 elements to find.

$T(n)=O(n/2)$

Worst Case:

If $x=41$ or 100 then we must examine all the 8 elements of the array. So the complexity is $O(n)$.

Binary search

Binary search method is also relatively simple method. For this method it is necessary to have the vector in an alphabetical or numerically increasing order. A search for a particular item with X resembles the search for a word in the dictionary. The approximate mid entry is located and its key value is examined. If the mid value is greater than X, then the list is chopped off at the $(\text{mid}-1)^{\text{th}}$ location. Now the list gets reduced to half the original list. The middle entry of the left-reduced list is examined in a similar manner. This procedure is repeated until the item is found or the list has no more elements. On the other hand, if the mid value is lesser than X, then the list is chopped off at $(\text{mid}+1)^{\text{th}}$ location. The middle entry of the right-reduced list is examined and the procedure is continued until desired key is found or the search interval is exhausted.

The algorithm for binary search is as follows,

Algorithm : binary search

Input : A, vector of n elements

K, search element

Output : low –index of k

Method : low=1,high=n

While($\text{low} \leq \text{high}-1$)

```
{
    mid=(low+high)/2
    if( $k < a[\text{mid}]$ )
        high=mid
    else
        low=mid
    if end
}
while end
if( $k = A[\text{low}]$ )
{
    write("search successful")
    write(k is at location low)
    exit();
}
else
    write (search unsuccessful);
if end;
algorithm ends.
```

Fibonacci search

The Fibonacci search technique is a method of searching a sorted array using a divide and conquer algorithm that narrows down possible locations with the aid of Fibonacci numbers. This method is faster than traditional binary search algorithms, with a complexity of $O(\log(x))$.

Algorithm

Let k be defined as an element in F, the array of Fibonacci numbers. $n = F_m$ is the array size. If the array size is not a Fibonacci number, let F_m be the smallest number in F that is greater than n.

The array of Fibonacci numbers is defined where $F_{k+2} = F_{k+1} + F_k$, when $k \geq 0$, $F_1 = 1$, and $F_0 = 0$.

To test whether an item is in the list of ordered numbers, follow these steps:

1. Set $k = m$.
2. If $k = 0$, stop. There is no match; the item is not in the array.

3. Compare the item against element in F_{k-1} .
4. If the item matches, stop.
5. If the item is less than entry F_{k-1} , discard the elements from positions $F_{k-1} + 1$ to n . Set $k = k - 1$ and return to step 2.
6. If the item is greater than entry F_{k-1} , discard the elements from positions 1 to F_{k-1} . Renumber the remaining elements from 1 to F_{k-2} , set $k = k - 2$, and return to step 2.

Pondicherry University Questions

2 Marks

1. What is an algorithm? (UQ April 2012 & APRIL 2013 Nov' 2016) (Ref.Pg.No.1 Qn.No.1)
2. What is best-case efficiency? (UQ APRIL 2013) (Ref.Pg.No.3 Qn.No.18)
3. What is binary search? (UQ April'12) (Ref.Pg.No.7 Qn.No.40)
4. Discuss about various asymptotic notation (UQ Nov'12) (Ref.Pg.No.8 Qn.No.45)
5. What is recursive algorithm? (UQ Nov'12, Apr/May'14) (Ref.Pg.No.8 Qn.No.46)
6. Define sorting (UQ Nov'12, Apr/May'14) (Ref.Pg.No.8 Qn.No.47)

11 Marks

1. Explain about analysis of algorithms & analysis of control structures. (UQ APRIL'13) (Ref.Pg.No.9 Qn.No.1)
2. Write about solving recurrences or Discuss about recursive and non recursive algorithms.(UQ APRIL12) (UQ:Apr/May'14 /May'16) (Ref.Pg.No.15 Qn.No.2)
3. Write and analyze about heap sort (UQ Nov'12) (Ref.Pg.No.30 Qn.No.3)
4. Write in detail about radix sort (UQ APRIL12) (Ref.Pg.No.33 Qn.No.4)
5. Write and analyze selection sort(UQ APRIL13 & Nov'12/Nov'16) (Ref.Pg.No.35 Qn.No.6)
6. Write and analyze about bubble sort (UQ APRIL 13) (Ref.Pg.No.37 Qn.No.7)
7. Write in detail about searching or Explain about Sequential and Fibonacci search (UQ APRIL13 & Nov'10) (UQ: Apr/May'14) (Ref.Pg.No.38 Qn.No.8)
8. Write and analyze Insertion sort.(May'16)